

Mercadillo Solidario

Documentación Técnica del Proyecto

Plataforma web para mercadillo de segunda mano con fines benéficos

Proyecto	Mercadillo Solidario
Versión	1.0.0
Centro	IES Albarregas
Ciclo	DAW2 – 2025/26

1. Descripción del Proyecto

Mercadillo Solidario es una aplicación web moderna desarrollada con React que permite gestionar un mercadillo de artículos de segunda mano con fines benéficos. La plataforma cuenta con un panel público donde cualquier usuario puede explorar y reservar productos, y un panel de administración protegido por autenticación para gestionar el catálogo completo.

Objetivos principales

- Ofrecer una experiencia de usuario moderna, accesible y responsive en todos los dispositivos.
- Proporcionar un catálogo dinámico con filtrado por categorías y búsqueda en tiempo real.
- Permitir a los visitantes reservar productos mediante un formulario de contacto.
- Dar a los administradores un CRUD completo para gestionar el inventario de productos.
- Garantizar la seguridad mediante autenticación con Supabase Auth y rutas protegidas.

Funcionalidades destacadas

Módulo	Descripción
Catálogo público	Visualización en grid de productos disponibles con imagen, precio y categoría.
Filtros y búsqueda	Filtrado por categoría (Libros, Ropa, Tecnología, Otros) y búsqueda por nombre.
Detalle de producto	Vista ampliada con toda la información y formulario de reserva.
Sistema de reservas	Formulario de contacto que almacena las reservas en base de datos.
Autenticación	Registro e inicio de sesión con email/contraseña mediante Supabase Auth.
Panel de administración	CRUD completo: crear, editar, cambiar estado y eliminar productos.

2. Tecnologías Empleadas

Stack tecnológico

Tecnología	Uso y versión
React 19	Librería principal para la construcción de la interfaz de usuario con componentes funcionales y hooks.
Vite 8	Herramienta de desarrollo y bundler ultra-rápido con HMR (Hot Module Replacement).
Tailwind CSS 4	Framework CSS utility-first para estilos responsivos. Complementado con CSS custom properties para el sistema de diseño.
Supabase	Backend-as-a-Service: base de datos PostgreSQL, autenticación (Auth) y API REST automática.
React Router v7	Enrutamiento client-side con rutas protegidas para el panel de administración.
Axios	Cliente HTTP para peticiones a la API.
Zod	Librería de validación de esquemas y tipos para TypeScript/JavaScript.
qrcode.react	Generación de códigos QR para productos (funcionalidad bonus).

Arquitectura del sistema

La aplicación sigue una arquitectura de tipo SPA (Single Page Application) con el siguiente flujo:

- El cliente (navegador) carga la aplicación React servida por Vite.
- React Router gestiona la navegación entre páginas sin recargas completas.
- Los componentes realizan peticiones directamente a la API de Supabase mediante su SDK oficial.
- Supabase gestiona la autenticación con JWT tokens y aplica Row Level Security (RLS) en la base de datos.
- Las variables de entorno (VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY) se configuran localmente o en el proveedor de hosting.

Base de datos

La base de datos PostgreSQL en Supabase contiene las siguientes tablas principales:

Tabla	Campos principales
productos	id, nombre, descripcion, precio, categoria, imagen_url, estado (Disponible/Reservado/Vendido), created_at
reservas	id, producto_id, nombre_usuario, email_usuario, mensaje, created_at

3. Estructura del Proyecto

```
mercadillo-solidario/
├── src/
│   ├── components/
│   │   ├── ui/           # Componentes reutilizables (Button, Card)
│   │   ├── Badge.jsx     # Badges de estado de productos
│   │   └── Navbar.jsx    # Barra de navegación
│   ├── context/
│   │   └── AuthContext.jsx # Contexto global de autenticación
│   ├── lib/
│   │   └── supabaseClient.js # Cliente Supabase inicializado
│   ├── pages/
│   │   ├── Catalog.jsx   # Catálogo público
│   │   ├── ProductDetail.jsx # Detalle + formulario de reserva
│   │   ├── Login.jsx     # Login y registro
│   │   └── AdminPanel.jsx # Panel de administración
│   ├── App.jsx           # Router principal y rutas protegidas
│   ├── main.jsx          # Punto de entrada
│   └── index.css         # Variables CSS y estilos globales
├── .env.local            # Variables de entorno (NO subir a git)
├── vite.config.js
└── package.json
```

4. Guía de Despliegue

Requisitos previos

- Node.js versión 20 o superior
- npm versión 10 o superior
- Cuenta en Supabase (supabase.com) – gratuita
- Cuenta en Vercel o Netlify para el hosting (opcionales)

4.0 Recursos del proyecto

- Aplicación desplegada: <https://mercadillo-solidario-olive.vercel.app/>
- Repositorio GitHub: https://github.com/JorgeAlegre-403/Mercadillo_Solidario.git

4.1 Instalación local

1. Clonar el repositorio:

```
git clone <url-repositorio> && cd mercadillo-solidario
```

2. Instalar las dependencias:

```
npm install
```

3. Crear el archivo de variables de entorno:

```
# Crear .env.local en la raíz del proyecto
VITE_SUPABASE_URL=https://tu-proyecto.supabase.co
VITE_SUPABASE_ANON_KEY=tu-clave-anon-aqui
```

4. Iniciar el servidor de desarrollo:

```
npm run dev
```

La aplicación estará disponible en ***http://localhost:5173***

4.4 Scripts disponibles

Comando	Descripción
npm run dev	Inicia el servidor de desarrollo con HMR en http://localhost:5173
npm run build	Genera la build de producción optimizada en la carpeta /dist
npm run preview	Previsualiza la build de producción localmente
npm run lint	Ejecuta ESLint para análisis estático del código

5. Instrucciones de Uso

5.1 Rutas de la aplicación

URL	Descripción
/	Catálogo público con todos los productos disponibles.
/producto/:id	Detalle de un producto específico con opción de reserva.
/login	Página de inicio de sesión y registro de administradores.
/admin	Panel de administración (requiere autenticación).

5.2 Catálogo público

Cualquier visitante puede acceder al catálogo sin necesidad de registrarse. Las acciones disponibles son:

- Explorar el grid de productos disponibles con imagen, nombre, descripción resumida y precio.
- Buscar productos por nombre utilizando el campo de búsqueda en tiempo real.
- Filtrar por categoría: Todos, Libros, Ropa, Tecnología u Otros.
- Hacer clic en "Ver Detalle" para acceder a la información completa del producto.

5.3 Reservar un producto

5. Navega al catálogo (/) y localiza el producto deseado.
6. Haz clic en "Ver Detalle" para abrir la página del producto.
7. Si el producto está disponible, aparecerá el botón "Reservar Este Producto".

8. Rellena el formulario con tu nombre, email y mensaje opcional.
9. Pulsa "Enviar Reserva". La solicitud queda registrada en la base de datos.
10. El administrador puede ver y gestionar las reservas desde el panel de administración.

5.4 Panel de administración

Acceso

El acceso al panel de administración requiere una cuenta de usuario. Para crear una:

11. Ve a /login y selecciona "¿No tienes cuenta? Regístrate".
12. Introduce tu email y una contraseña de al menos 6 caracteres.
13. Confirma tu email haciendo clic en el enlace que recibirás.
14. Inicia sesión para acceder al panel de administración en /admin.

Gestión de productos

- Crear producto: Pulsa "Nuevo Producto", rellena el formulario y guarda.
- Editar producto: Pulsa el botón "Editar" en la tarjeta del producto.
- Cambiar estado: Desde el formulario de edición, modifica el campo "Estado".
- Eliminar producto: Pulsa "Eliminar" y confirma en el diálogo de confirmación.

Campos del formulario de producto

Campo	Descripción y validación
Nombre	Obligatorio. Nombre descriptivo del artículo.
Categoría	Seleccionar entre: Libros, Ropa, Tecnología, Otros.
Descripción	Texto libre. Describe el estado y características del artículo.
Precio	Obligatorio. Precio sugerido en euros (acepta decimales).
Estado	Disponible / Reservado / Vendido. Controla la visibilidad en el catálogo.
URL Imagen	Opcional. URL pública de la imagen del producto.

6. Seguridad y Buenas Prácticas

Medidas de seguridad implementadas

- Autenticación mediante **Supabase Auth** con tokens **JWT** firmados.
- Row Level Security (RLS) en PostgreSQL para controlar el acceso a los datos por usuario.
- Variables de entorno para todas las credenciales sensibles (nunca hardcodeadas en el código).
- Archivo `.env.local` incluido en `.gitignore` para evitar exposición accidental de claves.
- Rutas protegidas en el frontend: el componente `ProtectedRoute` redirige a `/login` si no hay sesión activa.
- La clave anon de Supabase solo permite las operaciones definidas por las políticas RLS.

Recomendaciones adicionales para producción

- Configurar políticas RLS estrictas en Supabase para las tablas de reservas.
- Activar la confirmación de email en Supabase Auth para nuevos registros.
- Considerar el uso de Supabase Storage para almacenar imágenes en lugar de URLs externas.
- Añadir rate limiting en los formularios públicos para evitar spam de reservas.

7. Valoración del Trabajo Realizado

Logros conseguidos

El proyecto ha conseguido implementar con éxito todos los requisitos planteados inicialmente, alcanzando un nivel de calidad profesional tanto en el diseño visual como en la arquitectura del código.

- Se ha desarrollado una interfaz de usuario atractiva y coherente con un sistema de diseño propio basado en CSS custom properties, tipografías de Google Fonts (Fraunces + DM Sans) y un estilo neubrutalist moderno.
- La integración con Supabase ha permitido disponer de un backend completo sin necesidad de desarrollar una API propia, acelerando enormemente el desarrollo.
- El código está organizado siguiendo principios de separación de responsabilidades, con componentes reutilizables (Button, Card, Badge) y un contexto global para la autenticación.
- La aplicación es completamente responsive y funciona correctamente en dispositivos móviles, tablets y escritorio.

Dificultades encontradas

- La configuración de las políticas RLS en Supabase requirió varias iteraciones hasta obtener el comportamiento deseado, especialmente para diferenciar lecturas públicas de escrituras autenticadas.
- La gestión del estado de autenticación entre recargas de página y la sincronización con el contexto de React presentó algunos desafíos iniciales que se resolvieron con useEffect y los listeners de Supabase.
- La compatibilidad entre Tailwind CSS v4 y el sistema de estilos custom requirió un enfoque híbrido, usando Tailwind principalmente para layout y las CSS custom properties para el sistema de colores y tipografía.

Posibles mejoras futuras

- Integración de Supabase Storage para la subida directa de imágenes desde el panel de administración.
- Implementar notificaciones por email al administrador cuando se realiza una nueva reserva.
- Añadir paginación o scroll infinito en el catálogo para mejorar el rendimiento con muchos productos.
- Panel de gestión de reservas dentro del área de administración.
- Internacionalización (i18n) para soporte multi-idioma.
- Tests unitarios y de integración con Vitest y React Testing Library.
- Modo oscuro (dark mode) basado en las preferencias del sistema operativo.

Valoración global

El proyecto representa una aplicación web full-stack funcional y bien estructurada, que demuestra el dominio de tecnologías modernas del ecosistema JavaScript/React. La

elección de Supabase como backend agilizó el desarrollo y permitió centrarse en la experiencia de usuario, que es uno de los puntos fuertes del proyecto.

La arquitectura elegida es escalable y permitiría añadir nuevas funcionalidades sin necesidad de reestructurar el código base. El uso de React Router v7, el manejo del estado con hooks nativos de React y la integración con el SDK oficial de Supabase siguen las mejores prácticas actuales del ecosistema.

En resumen, se trata de un proyecto sólido que cumple los objetivos planteados, con un diseño visual diferenciado y una base técnica robusta para continuar su desarrollo.